Unit-02

decision control statements

`if`, `if-else`, and `if-elif-else` **statements. They help in executing specific blocks of code based on the evaluation of conditions.**

1. if Statement

Syntax:

```
if condition:
    # Code
```

Example:

```
x = 10
if x > 5:
    print("x is greater than 5")
Output: x is greater than 5
```

---

2. `if-else` Statement

Syntax:

```
if condition:
    # Code to execute if condition is True
else:
    # Code to execute if condition is False
```

Example:

```
x = 3
if x > 5:
    print("x is greater than 5")
else:
    print("x is less than or equal to 5")
# Output: x is less than or equal to 5
```

---

3. `if-elif-else` Statement

Syntax:

```
if condition1:
    # Code if condition1 is True
```

```
elif condition2:
    # Code if condition2 is True
elif condition3:
    # Code if condition3 is True
else:
    # Code if none of the conditions are True
```

Example:

```
x = 15
if x < 10:
    print("x is less than 10")
elif x == 10:
    print("x is equal to 10")
else:
    print("x is greater than 10")
# Output: x is greater than 10
```

---

4. `pass` in `if` Statements

The `pass` statement is a placeholder when you need an empty block of code.

Example:

```
x = 10
if x > 5:
    pass  # Placeholder for future code
else:
    print("x is less than or equal to 5")
```

---

identity operators **and** membership operators

1. Identity Operators

Identity operators compare the memory location of two objects. They are used to determine whether two variables refer to the same object in memory.

Operators

- **`is`**: Returns `True` if two variables point to the same object (same memory location).
- **`is not`**: Returns `True` if two variables do not point to the same object (different memory locations).

Examples

```
x = [1, 2, 3]
y = [1, 2, 3]
z = x

# `is` checks identity
print(x is z)        # True, `z` refers to the same object as `x`
print(x is y)        # False, `x` and `y` have the same value but are
different objects

# `is not` checks non-identity
print(x is not y)    # True, `x` and `y` are different objects
```

Difference Between `is` and `==`

```
x = [1, 2, 3]
y = [1, 2, 3]

print(x == y)        # True, values are equal
print(x is y)        # False, different objects in memory
```

2. Membership Operators

Membership operators are used to check whether a value is present in a sequence (e.g., strings, lists, tuples, sets, dictionaries).

Operators

- **`in`**: Returns `True` if the value is found in the sequence.
- **`not in`**: Returns `True` if the value is not found in the sequence.

Key Points

1. These operators work with any **iterable**, such as:
    - Strings
    - Lists
    - Tuples
    - Sets
    - Dictionaries (keys only)
2. Membership operators are case-sensitive.

Examples

With Strings:

```
text = "Python programming"
print("Python" in text)          # True, "Python" is a substring
print("python" in text)          # False, case-sensitive
print("Java" not in text)        # True, "Java" is not a substring
```

With Lists:

```
numbers = [10, 20, 30, 40]
print(20 in numbers)             # True, 20 is in the list
print(50 not in numbers)         # True, 50 is not in the list
```

Practice Problems

1. **Check Even or Odd:**
   Write a program to check if a number entered by the user is even or odd.
   Hint: Use the modulus operator `%`.

2. **Find Maximum Number:**
   Write a program to input three numbers and print the larger number.
   Hint: Use an `if-else` statement.

3. **Check Positive or Negative:**
   Write a program to check if a number entered by the user is positive, negative, or zero.

4. **Leap Year Check:**
   Write a program to determine if a year entered by the user is a leap year.
   Hint: A year is a leap year if:
   - Divisible by 4 but not divisible by 100, or
   - Divisible by 400.

5. **Pass or Fail:**
   Write a program to input marks of a student and print "Pass" if the marks are 40 or more, otherwise print "Fail."

6. **Divisibility Check:**
   Write a program to check if a number is divisible by both 3 and 5.
   Hint: Use `and` in the condition.

7. **Vowel or Consonant:**
   Write a program to check if a given letter is a vowel or a consonant.
   Hint: Use `in` with strings like `"aeiou"`.

9. **Eligibility to Vote:**
   Write a program to check if a person is eligible to vote based on their age (18 or older).

11. **Triangle Validity Check:**
    Write a program to check if three sides entered by the user can form a triangle.
    Hint: The sum of any two sides must be greater than the third sid

12. **Grade Calculation:**
    Write a program to assign grades based on marks:

- Marks >= 90: A
- Marks >= 80: B
- Marks >= 70: C

- Marks >= 60: D
- Otherwise: F

14. **Find the Largest Among Three Numbers:**
    Write a program to input three numbers and find the largest number using nested `if-else`.

15. **Calculate Electricity Bill:**
    Write a program to calculate electricity charges:

- First 100 units: ₹5/unit
- Next 200 units: ₹7/unit
- Beyond 300 units: ₹10/unit

18. **Day of the Week:**
    Write a program to input a number (1-7) and print the corresponding day of the week (e.g., 1 for Monday, 2 for Tuesday, etc.).
    Hint: Use `elif`.

19. **Simple Calculator:**
    Write a program to perform addition, subtraction, multiplication, or division based on user input.
    Hint: Use `if-elif-else` with a menu-driven approach.

20. **Check Triangle Type:**
    Write a program to determine the type of triangle:

- Equilateral (all sides equal),
- Isosceles (two sides equal),
- Scalene (all sides different).

21. **BMI Calculator:**
    Write a program to calculate Body Mass Index (BMI) and determine the category:

- BMI < 18.5: Underweight
- BMI 18.5 - 24.9: Normal
- BMI 25.0 - 29.9: Overweight
- BMI >= 30: Obese

22. **Check Divisibility by Multiple Numbers:**
    Write a program to check if a number is divisible by 4, 6, or 9.
    Hint: Use multiple conditions.

23. **Nested If for Age Check:**
    Write a program that:

- If age is greater than or equal to 18, check if the person is also eligible to drive.
- If the age is less than 18, print "Too young."

---

loops

loops are used to execute a block of code repeatedly, either for a fixed number of times or until a specific condition is met.

1. **`for` loop**
2. **`while` loop**

1. `while` Loop

A **`while` loop** repeatedly executes a block of code as long as a specified condition is true. Once the condition becomes **false**, the loop terminates and the program continues with the next line of code.

Syntax:

```
while condition:
    # code block to execute
```

Example 1: Basic `while` Loop

```
i = 1  # Initialize variable
while i <= 5:  # Loop condition
    print(i)  # Print the value of i
    i += 1  # Increment i by 1
```

Output:

```
1
2
3
4
5
```

Ques: Write a Python program that uses a `while` loop to display the table?

Example 2: Infinite `while` Loop

If the condition is always `True`, the loop will run indefinitely (infinite loop), unless a `break` statement is used to exit the loop manually.

```
i = 1
while True:
    print(i)
    i += 1
    if i > 5:  # Breaking the loop after 5 iterations
        break
```

Output:

```
1
2
3
4
5
```

Example 3: Using `break` and `continue`

```
i = 0
while i < 10:
    i += 1
    if i == 5:
        continue  # Skip the rest of the loop when i is 5
    if i == 8:
        break  # Exit the loop when i is 8
    print(i)
```

Output:

```
1
2
3
4
6
7
```

Example 4: `else` with `while` loop

The `else` block in a `while` loop is executed only if the loop completes **normally** (not interrupted by `break`).

```
i = 1
while i <= 5:
    print(i)
    i += 1
else:
    print("Loop completed normally.")
```

Explanation:

- The loop runs while `i <= 5`, printing the values.
- Once the loop finishes (without `break`), the `else` block runs.

Output:

```
1
2
```

```
3
4
5
Loop completed normally.
```

Practice Problems:

1) **Sum of Natural Numbers**
   Hint: Use a `while` loop with a variable initialized to `1`. Keep adding it to a sum variable and increment the counter until it reaches `n`.
2) **Factorial of a Number**
   Hint: Use a `while` loop starting from `1` up to `n`. Multiply all the numbers in each iteration.
3) **Multiplication Table**
   Hint: Use a `while` loop to multiply a fixed number (entered by the user) with a counter variable that starts at `1` and goes up to `10`.
4) **Count Digits of a Number**
   Hint: Use `while number > 0` to repeatedly divide the number by `10` (integer division) and count how many times the loop executes.
5) **Reverse a Number**
   Hint: Use `while number > 0` to extract the last digit using `% 10`, then construct the reversed number by multiplying the result by `10` and adding the digit.
6) **Check if a Number is a Palindrome**
   Hint: Reverse the number and compare it with the original number.
7) **Armstrong Number**
   Hint: Use `while` to separate the digits of the number, cube them, and check if their sum equals the original number.
8) **Prime Numbers Between Two Numbers**
   Hint: Use nested loops. The outer `while` loop goes through numbers between the range, and the inner loop checks if the current number has any divisors.
9) **Fibonacci Sequence**
   Hint: Use two variables to store the current and previous terms. In each iteration, calculate the next term as the sum of these two and update them.
10) **Guess the Number**
    Hint: Use `while True` to keep asking the user for a guess. Compare the guess with the random number and provide feedback until the user guesses correctly.

    Ans:

```
import random
random_number = random.randint(1, 3)
print("Welcome to the 'Guess the Number' game!")
print("I have picked a number between 1 and 3. Can you guess it?")

while True:
    # Get user input as an integer
    user_guess = int(input("Enter your guess: "))

    if user_guess < random_number:
        print("Too low! Try again.")
    elif user_guess > random_number:
```

```
    print("Too high! Try again.")
  else:
    print(f"Congratulations! You guessed it right. The number was {random_number}.")
    break
```

Nested Loops

Nested loop is a loop placed inside another loop. Both `while` and `for` loops can be nested, and they allow you to solve problems requiring iteration over multiple dimensions.

Nested loops involve two or more loops:

1. **Outer Loop**: Controls the overall structure or rows.
2. **Inner Loop**: Executes completely for each iteration of the outer loop.

Syntax

```
while outer_condition:   # Outer loop
    # Code for the outer loop
        while inner_condition:  # Inner loop
            # Code for the inner loop
```

The **inner loop** executes fully for each iteration of the **outer loop**.

Example 1: Basic Nested Loop

```
i = 1
while i <= 3:  # Outer loop
    j = 1
    while j <= 2:  # Inner loop
        print(f"i = {i}, j = {j}")
        j += 1
    i += 1
```

Output:

```
i = 1, j = 1
i = 1, j = 2
i = 2, j = 1
i = 2, j = 2
i = 3, j = 1
i = 3, j = 2
```

Example 2: Generating Patterns

```
*****
*****
*****
*****
n = 4
```

```
i = 1
while i <= n:  # Outer loop for rows
    j = 1
    while j <= n:  # Inner loop for columns
        print("*", end="")  # Print star in the same line
        j += 1
    print()  # Move to the next line
    i += 1
```

Example 3: Pyramid Patterns

```
    *
   ***
  *****
 *******
*********
n = 5  # Number of rows
i = 1
while i <= n:
    # Print spaces
    j = 1
    while j <= n - i:
        print(" ", end="")
        j += 1
    # Print stars
    k = 1
    while k <= 2 * i - 1:
        print("*", end="")
        k += 1
    print()  # Move to the next line
    i += 1
```

Example 4: Number Patterns

```
1
12
123
1234
12345
n = 5  # Number of rows
i = 1
while i <= n:
    j = 1
    while j <= i:
        print(j, end="")  # Print numbers in the same line
        j += 1
    print()  # Move to the next line
    i += 1
```

---

2. `for` Loop

The `for` loop is used to iterate over a sequence (like a list, tuple, string, or range).

Syntax:

```
for variable_name in sequence:
    # Code
```

Example 1: Using `range()` with a `for` Loop

```
# Print numbers from 1 to 5
for i in range(1, 6):
    print(i)
# Output:
# 1
# 2
# 3
# 4
# 5

# Print even numbers from 2 to 10
for i in range(2, 11, 2):
    print(i)
# Output:
# 2
# 4
# 6
# 8
# 10
```

Example 2: Iterating Over a String

```
word = "Python"

for letter in word:
    print(letter)
# Output:
# P
# y
# t
# h
# o
# n
```

Example 3: Nested `for` Loops

```
for i in range(1, 4):  # Outer loop
    for j in range(1, 4):  # Inner loop
        print(f"i = {i}, j = {j}")
# Output:
# i = 1, j = 1
# i = 1, j = 2
# i = 1, j = 3
# i = 2, j = 1
# i = 2, j = 2
# i = 2, j = 3
# i = 3, j = 1
```

```
# i = 3, j = 2
# i = 3, j = 3
```
